

编译原理 2025 风格模拟卷06（题目与考试标准答案）

卷面说明

本卷按 Slides/Slides/handout 与三份 Recap PPT 更新，主线覆盖前端自动机与 LL、中端 IR/SSA、数据流、符号执行、后端目标代码、寄存器分配、指令调度以及过程间/指针分析。总分 100 分。

A 卷题目

一、词法分析（15 分）

正则表达式：

```
(epsilon|a) b (b|c)*
```

1. 使用 Thompson 构造法构造等价 NFA。
2. 使用子集构造法构造 DFA，写明 DFA 状态含义。
3. 最小化 DFA。

二、LL 语法分析（15 分）

文法：

```
E -> F G  
F -> F bang | int  
G -> times | slash
```

1. 消除左递归。
2. 写 FIRST 和 FOLLOW。
3. 写预测分析表。
4. 判断消除左递归后的文法是否为 LL(1)。

三、指令调度（15 分）

给定指令：

```
I1: R1 = R2 + R3  
I2: R4 = R1 + R5  
I3: R2 = R6 + R7  
I4: R1 = R8 + R9  
I5: R10 = R4 + R1
```

1. 写出指令调度的目的。
2. 写出 RAW、WAR、WAW 的定义。
3. 构建依赖图，WAR/WAW 用虚线表示。
4. 在允许寄存器重命名的条件下给出一个合法重排序列。

四、支配问题（15 分）

CFG: while 循环

```
Entry->B1  
B1->B2  
B2->B3  
B3->B2  
B2->Exit
```

1. 写出 Dom、PostDom、Dom Frontier 的定义。
2. 计算每个节点的 Dom 集和 idom。
3. 写出支配边界。
4. 说明 SSA 中 phi 函数应放在什么位置。

五、符号执行与 SAT（20 分）

公式：

```
F = ((a6 and b6) or c6)
```

1. 写出 Path-sensitive、Flow-sensitive、Context-sensitive 的定义。
2. 写出 CNF 的定义。
3. 使用 Tseitin 算法把 F 转成 CNF。
4. 证明 2025 卷中的两组公式同时可满足。

六、Andersen 指针分析（20 分）

程序：

```
x = &o1
y = &o2
z = x
*z = y
w = *x
```

1. 对每条语句写 Andersen 约束。
2. 根据约束构建闭包。
3. 写出最终 points-to 集合。

七、PPT 新增重点：过程间分析（10 分）

程序片段：

```
id(x) { return x; }
a = id(1);
b = id(2);
```

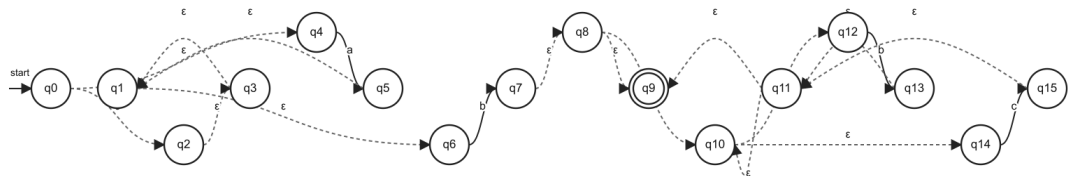
1. 说明过程间分析为什么需要区分调用上下文。
2. 写出 context-insensitive 分析下 **id** 返回值可能是什么。
3. 写出 1-call-string context-sensitive 分析下两个调用点的返回值。

B 按考试标准重写答案

一、词法分析答案

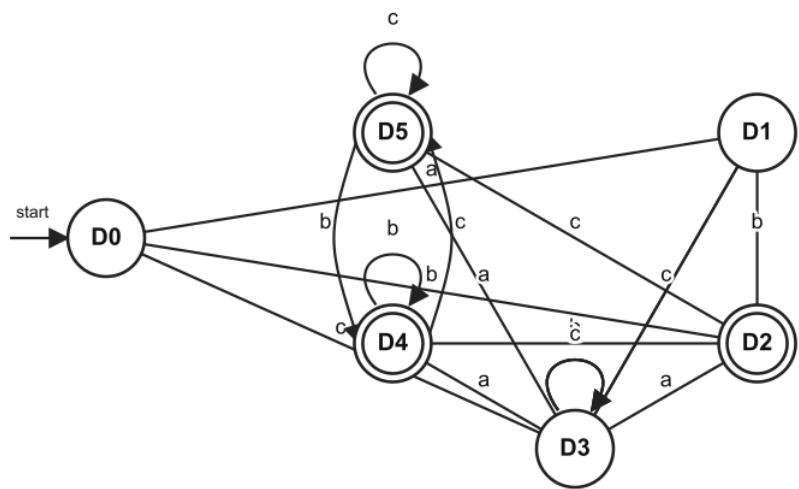
本题图形化答案如下，三张图分别对应 Thompson NFA、子集构造 DFA 和最小 DFA。

Thompson NFA: $(\epsilon|a) b (b|c)^*$



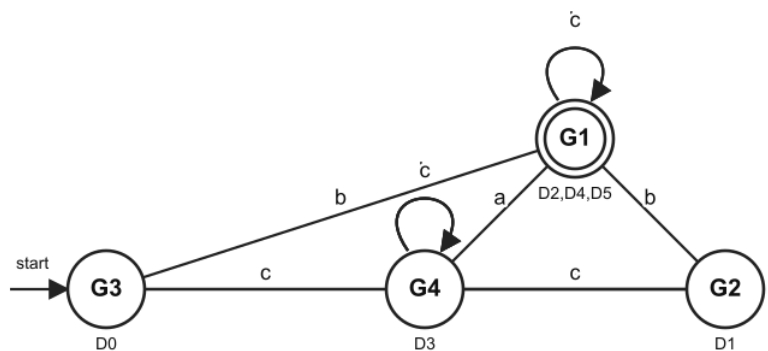
模拟卷06 Thompson NFA

Subset DFA: $(\epsilon|a) b (b|c)^*$



模拟卷06 子集构造 DFA

Minimal DFA: $(\epsilon|a) b (b|c)^*$



模拟卷06 最小 DFA

正则表达式: $(\epsilon|a) b (b|c)^*$

Thompson NFA: 起点 q0, 终点 q9。按下列转移表画图, q9 画双圈。

```
q2 --epsilon--> q3
q4 --a--> q5
q0 --epsilon--> q2
q0 --epsilon--> q4
q3 --epsilon--> q1
q5 --epsilon--> q1
q6 --b--> q7
q1 --epsilon--> q6
q12 --b--> q13
q14 --c--> q15
q10 --epsilon--> q12
q10 --epsilon--> q14
q13 --epsilon--> q11
q15 --epsilon--> q11
q8 --epsilon--> q10
q8 --epsilon--> q9
q11 --epsilon--> q10
q11 --epsilon--> q9
q7 --epsilon--> q8
```

子集构造 DFA:

D0:

NFA集合 = {q0,q1,q2,q3,q4,q6}

on a -> D1

on b -> D2

on c -> D3

终态 = 否

D1:

NFA集合 = {q1,q5,q6}

on a -> D3

on b -> D2

on c -> D3

终态 = 否

D2:

NFA集合 = {q7,q8,q9,q10,q12,q14}

on a -> D3

on b -> D4

on c -> D5

终态 = 是

D3:

NFA集合 = EMPTY

on a -> D3

on b -> D3

on c -> D3

终态 = 否

D4:

NFA集合 = {q9,q10,q11,q12,q13,q14}

on a -> D3

on b -> D4

on c -> D5

终态 = 是

D5:

NFA集合 = {q9,q10,q11,q12,q14,q15}

on a -> D3

on b -> D4

on c -> D5

终态 = 是

最小化:

初分组:

终态组 = {D2,D4,D5}

非终态组 = {D0,D1,D3}

最终分组:

G1 = {D2,D4,D5}

G2 = {D1}

G3 = {D0}

G4 = {D3}

每个最终分组对应一个最小 DFA 状态。

二、LL 语法分析答案

原文法:

E -> F G

F -> F bang | int
G -> times | slash

消除左递归：
E -> F G
F -> int F'
F' -> bang F' | epsilon
G -> times | slash

FIRST:
FIRST(E)={ int }
FIRST(F)={ int }
FIRST(F')={ bang, epsilon }
FIRST(G)={ times, slash }

FOLLOW:
FOLLOW(E)={ \$ }
FOLLOW(F)={ times, slash }
FOLLOW(F')={ times, slash }
FOLLOW(G)={ \$ }

预测表：
M[E,int] = E -> F G
M[F,int] = F -> int F'
M[F',bang] = F' -> bang F'
M[F',times] = F' -> epsilon
M[F',slash] = F' -> epsilon
M[G,times] = G -> times
M[G,slash] = G -> slash

结论：表中无多重入口，消除左递归后的文法是 LL(1)。

预测分析表（表格形式）：

非终结符	输入符号	产生式
E	int	E -> F G
F	int	F -> int F'
F'	bang	F' -> bang F'
F'	times	F' -> epsilon
F'	slash	F' -> epsilon
G	times	G -> times
G	slash	G -> slash

三、指令调度答案

定义：

指令调度在不改变程序语义的前提下重排指令，减少流水线停顿，提高并行执行效率。
RAW：先写后读，必须保持顺序。
WAR：先读后写，写者不能提前。
WAW：先写后写，顺序影响最终值。

指令：
I1: R1 = R2 + R3
I2: R4 = R1 + R5
I3: R2 = R6 + R7
I4: R1 = R8 + R9
I5: R10 = R4 + R1

依赖：
I1 -> I2 RAW on R1
I1 -> I3 WAR on R2
I1 -> I4 WAW on R1
I2 -> I4 WAR on R1
I2 -> I5 RAW on R4
I4 -> I5 RAW on R1

寄存器重命名：
I3: R2_new = R6 + R7
I4: R1_new = R8 + R9
I5 使用 R1_new

重命名后保留 RAW，WAR/WAW 被消除。一个合法调度序列：
I1, I3, I4, I2, I5

四、支配问题答案

定义：

Dom(n)：入口到 n 的每条路径都经过的节点集合。
PostDom(n)：n 到出口的每条路径都经过的节点集合。
Dom Frontier(n)：n 支配某个前驱，但不严格支配该节点的位置。

CFG 边：
Entry->B1
B1->B2
B2->B3
B3->B2
B2->Exit

Dom 集：
Dom(Entry)={Entry}
Dom(B1)={Entry,B1}
Dom(B2)={Entry,B1,B2}
Dom(B3)={Entry,B1,B2,B3}
Dom(Exit)={Entry,B1,B2,Exit}

直接支配者：
idom(B1)=Entry
idom(B2)=B1
idom(B3)=B2
idom(Exit)=B2

支配边界：
DF(B2)={B2}
DF(B3)={B2}
其余节点 DF 为空集

SSA phi 放置：若变量在多个基本块中被定义，则在这些定义块的迭代支配边界处插入该变量的 phi。直观上，phi 放在多条控制流汇合且不同定义可能到达的位置。

五、符号执行与 SAT 答案

公式：F = ((a6 and b6) or c6)
引入：x61 <-> (a6 and b6), x62 <-> (x61 or c6)，根变量 x62 为真。

CNF：
(not x61 or a6)
(not x61 or b6)
(x61 or not a6 or not b6)
(x62 or not x61)
(x62 or not c6)
(not x62 or x61 or c6)
(x62)

敏感性定义：
Path-sensitive 区分不同路径。
Flow-sensitive 区分程序点和语句顺序。
Context-sensitive 区分函数调用上下文。

同时可满足证明套用：
F1=(a0 or ... or an or c) and (b0 or ... or bm or not c), F2=(a0 or ... or an or b0 or ... or bm)。
F1 为真时，c=false 推出某个 ai=true, c=true 推出某个 bj=true，所以 F2 为真。
F2 为真时，若某个 ai=true 令 c=false；若某个 bj=true 令 c=true，所以 F1 为真。

六、Andersen 指针分析答案

程序:

```
x = &o1
y = &o2
z = x
*z = y
w = *x
```

约束:

```
o1 in pts(x)
o2 in pts(y)
pts(x) subset pts(z)
for v in pts(z), pts(y) subset pts(v)
for v in pts(x), pts(v) subset pts(w)
```

闭包:

```
pts(x)={o1}
pts(y)={o2}
pts(z)={o1}
由 *z=y 得 pts(o1)={o2}
由 w=*x 得 pts(w)={o2}
```

七、PPT 新增重点答案

过程间分析跨函数传播事实。

如果不区分调用上下文，不同调用点的信息会合并，精度下降。

context-insensitive:

id 的参数事实来自两个调用点，会合并为 {1,2}。

因此返回值也可能是 {1,2}。

a 可能为 {1,2}，b 可能为 {1,2}。

1-call-string context-sensitive:

调用点 a=id(1) 的上下文中，id 返回 1。

调用点 b=id(2) 的上下文中，id 返回 2。

因此 a={1}, b={2}。